5. Build and maintenance manual

This manual explains, for developers, how to build QCalc again from the sources in the repository (the test APK and the App Bundle for Google Play), how to publish and update it on Google Play, and what to watch for in maintenance.

5.1 Development environment

The script builds were checked on Ubuntu 24.04 (64-bit x86-64; checked on Ubuntu 24.04.5 LTS). The Android NDK is not used. The same sources can also be opened in Android Studio (Gradle) (5.11).

```
sudo apt install build-essential autoconf automake libtool intltool autopoint
gettext pkg-config \
  aapt dalvik-exchange zipalign apksigner openjdk-17-jdk-headless git curl
xz-utils qemu-user-static
pip install ziglang==0.17.1
# The platform files of the Android SDK (API 36 and API 34): with the SDK
Manager of Android Studio, or this command
sdkmanager "platforms;android-36" "platforms;android-34"
# Only to make the icons again
sudo apt install librsvg2-bin imagemagick
```

Tool	Version checked	Used for
zig (ziglang from pip)	0.17.1	Cross-compiling C/C++ for aarch64-linux-musl
aapt2 (Ubuntu's aapt package)	1:14~beta1 (aapt2 2.19)	Resources and the manifest: for the APK, and in protobuf format for the App Bundle
android.jar (Android SDK Platform 36)	API 36	Compiling Java. In $ANDROID_HOME/platforms/android-36, or set with the environment variable ANDROID_JAR
android.jar (Android SDK Platform 34)	API 34	Linking with aapt2, as Ubuntu's aapt2 cannot read the resource tables of API 35 and later. Can be set with the environment variable AAPT2_JAR
JDK (javac, jarsigner, keytool)	OpenJDK 21.0.12 (javac also checked with 17)	Compiling in Java 8 style (-source 8 -target 8), signing the App Bundle, making keys

Tool	Version checked	Used for
dalvik-exchange (dx)	10.0.0	Turning class files into classes.dex
zipalign, apksigner	10.0.0, 31.0.2	Aligning and signing the test APK
python3	3.13 (standard library only)	Putting the App Bundle together (build-aab.sh), making the license texts (make-licenses.py)
rsvg-convert, ImageMagick	–	Only to make the icons again (icon/make-icons.sh). The icons made are in app/res and store/
qemu-aarch64-static	8.2.2	Running the arm64 engine on a PC for tests
bundletool (optional)	1.17.2	Checking the App Bundle, and that device APKs can be made from it (5.6)
Android Studio (optional)	A version for Android Gradle Plugin 8.11.1 and Gradle 8.13	Building from the same sources and running on a phone (5.11). It does not build the engine

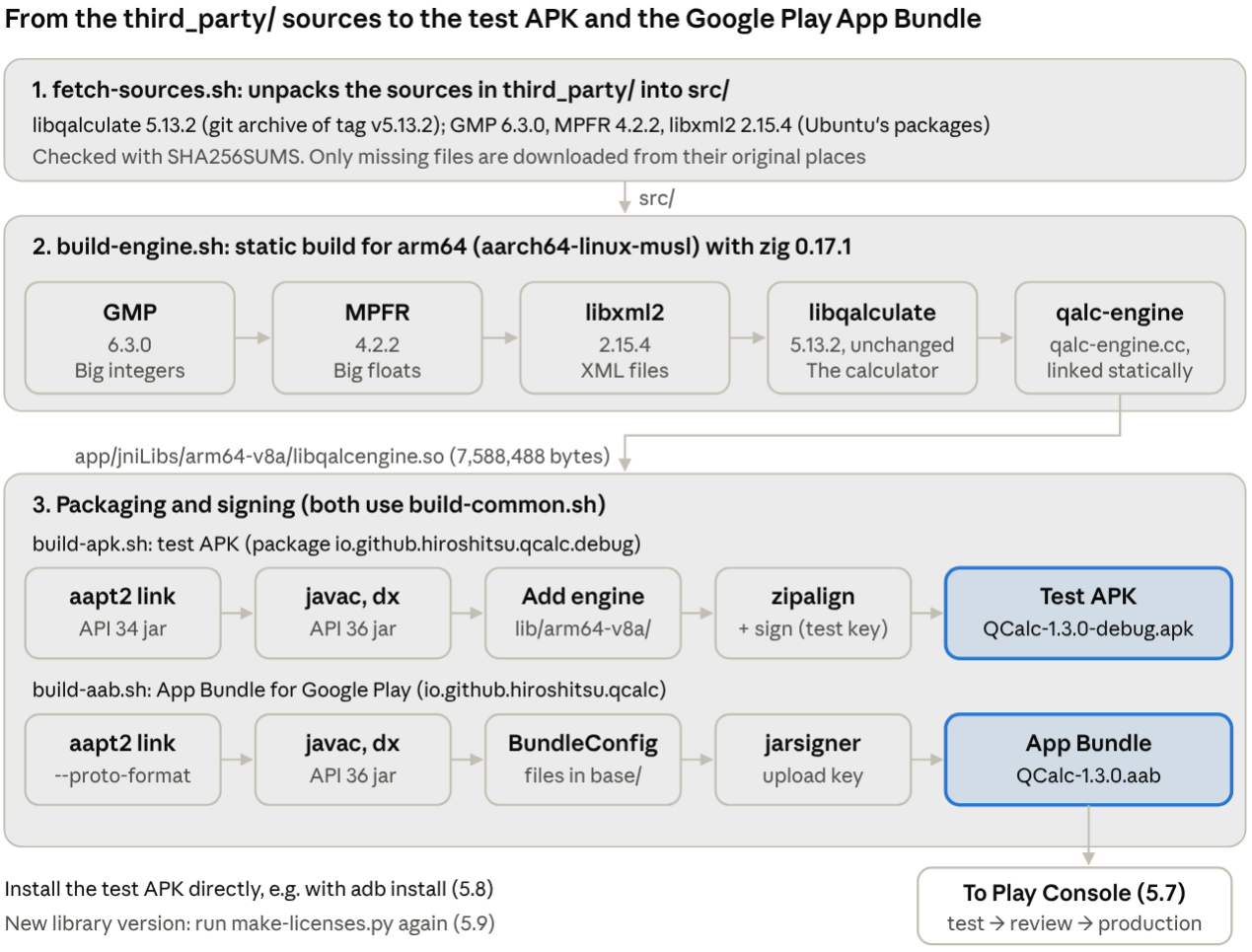
Folders

Folder or file	Contents	In the repository
app/	AndroidManifest.xml, the Gradle settings (build.gradle), Java (the 5 files in src/io/github/hiroshitsu/qcalc/; PlayBilling, only in versions with billing, is in src-billing/), resources (res/; the test build's name is in res-debug/), the page (assets/, with licenses.js) and the built engine (jniLibs/arm64-v8a/libqalcengine.so)	Yes
bridge/	qalc-engine.cc (the engine program). Building makes qalc-engine, which is also copied to app/jniLibs	qalc-engine.cc only

Folder or file	Contents	In the repository
third_party/	The sources of the libraries built into the engine (tar.xz of libqalculate, GMP, MPFR and libxml2), SHA256SUMS, README.md	Yes (the corresponding sources for the GPL and LGPL)
icon/	The icon's source (qcalc-icon.svg) and make-icons.sh, which makes the icons in app/res and store/icon-512.png	Yes
fetch-sources.sh, build-engine.sh, build-common.sh, build-apk.sh, build-aab.sh, make-licenses.py, version.properties	The build scripts and the version of the app (shared by all builds)	Yes
settings.gradle, build.gradle, gradle.properties, gradle/wrapper/, keystore.properties.example	Settings for Android Studio (Gradle), and a sample of the signing settings	Yes
docs/	The pages published on GitHub Pages (introduction page, privacy policy; English first, then Japanese)	Yes
design/	PDFs of the design documents and manuals (Japanese and English), and their list (README.md)	Yes
store/	The Google Play listing texts (listing-en.md, listing-ja.md), icon, feature graphic and screenshots (screenshots-en/ in English, screenshots/ in Japanese)	Yes
README.md (English), README.ja.md (Japanese), LICENSE, .gitignore	Description, the full text of GNU GPL v2, the list of what git does not track	Yes
src/	Sources unpacked from third_party/	No (made by the scripts)

Folder or file	Contents	In the repository
tools/, build/, prefix/, sdk/, Gradle's work folders (.gradle/, build-gradle/, app/build/)	Scripts that call zig, work folders, the static libraries, the build output of Android Studio	No
out/	The APKs and App Bundles made	No
signing/, keystore.properties	The upload key (qcalc-upload.jks) and the file with its password, the test key (qcalc-debug.keystore), the settings for signing in Android Studio	Never (excluded by .gitignore and kept elsewhere)

5.2 How the build goes



Build flow · the test APK and the App Bundle in 3 stages

Three stages of scripts make the test APK and the App Bundle for Google Play. build-engine.sh skips libraries already built (the .a files in prefix/lib), so after changing only the page or Java, run just build-apk.sh or build-aab.sh, and after changing qalc-engine.cc, start from build-engine.sh. The repository includes the built engine (app/jniLibs), so as long as the engine does not change, neither zig nor fetch-sources.sh is needed. Building with Android Studio is in 5.11.

```
./fetch-sources.sh        # unpacks the sources of third_party/ into src/
(the first time)
./build-engine.sh         # builds the libraries and the engine, and copies
it to app/jniLibs
./build-apk.sh            # test APK: out/QCalc-1.3.0-debug.apk
./build-aab.sh            # for Google Play: out/QCalc-1.3.0.aab (needs the
upload key)
python3 make-licenses.py  # only after changing a library version:
app/assets/licenses.js
```

5.3 Preparing the sources (fetch-sources.sh and third_party/)

Source	Version	File in third_party/	Original source
libqalculate	5.13.2	libqalculate-5.13.2.tar.xz	A git archive of the tag v5.13.2 (commit d3672af9cf6e05a39bd4206b7b7d6b1e0d194caa) of https://github.com/Qalculate/libqalculate
GMP	6.3.0	gmp_6.3.0+dfsg.orig.tar.xz	https://archive.ubuntu.com/ubuntu/pool/main/g/gmp/
MPFR	4.2.2	mpfr4_4.2.2.orig.tar.xz	https://archive.ubuntu.com/ubuntu/pool/main/m/mpfr4/
libxml2	2.15.4	libxml2_2.15.4+dfsg.orig.tar.xz	https://archive.ubuntu.com/ubuntu/pool/main/libx/libxml2/

- It unpacks the tar.xz files in third_party/ into src/. Only missing files are downloaded from their original sources (for libqalculate, the tag v5.13.2 is cloned with git and packed with git archive).
- The 3 files from Ubuntu are checked with SHA256SUMS (sha256sum -c), and the script stops if they do not match.
- The sources in third_party/ are there to publish in the repository the “corresponding source” that the GPL and LGPL require. When a version changes, replace the file, SHA256SUMS and README.md too (5.9).
- The GMP source that Ubuntu distributes has no folder of manuals, so an empty doc/Makefile.in, which configure needs, is made.

- The configure.ac of the upstream tag v5.13.2 still says 5.13.1 (and qalc --version shows 5.13.1). The version that the app shows is QALC_ENGINE_VERSION ("5.13.2"), passed by build-engine.sh.

5.4 Building the calculation engine (build-engine.sh)

zig is used as the cross compiler to make an arm64 executable with everything linked statically. It depends on no Android library, so the Android NDK is not needed.

1. Makes the scripts that call zig in tools/ (zcc, zcxx, zar and zranlib, for aarch64-linux-musl).
2. Builds GMP → MPFR → libxml2 → libqalculate, in this order, as static libraries (-O2) into prefix/. A library whose .a is in prefix/lib is skipped, so delete that .a to build it again.
3. Compiles bridge/qalc-engine.cc as C++17 and links everything statically (-static; -s strips the symbols; -Wl,-z,max-page-size=16384 supports devices with 16 KB pages). The result is bridge/qalc-engine (7,588,488 bytes in 1.3.0), which is also copied to app/jniLibs/arm64-v8a/libqalcengine.so to go into the APK and the App Bundle.

configure options of libqalculate

Option	Reason
--enable-compiled-definitions	Builds the definitions of units, functions and the rest into the program (no definition files to ship)
--disable-insecure	Turns off command(), which runs commands, and uptime
--without-libcurl	No networking (the app's Java gets the exchange rates)
--without-icu	No ICU
--without-gnuplot-call	Does not call gnuplot (the page draws the graphs)
--with-readline=no	No line editing for interactive use
--disable-nls	No translations (the messages are in English; on the Japanese screen the page translates the common ones)

libxml2 is built without python, zlib, lzma, ICU, HTTP, debugging and catalogs. The log of each step stays in configure.log and make.log in build/〈library name〉/, and their end is shown when a step fails.

5.5 Making the test APK (build-apk.sh)

1. Resources (app/res): the app name (QCalc), the light and dark themes (values-night), and the adaptive icon for Android 8.0 and later (with a monochrome layer for themed icons). For the test build, app/res-debug changes the app name to “QCalc (debug)”.
2. Adds the package attribute (io.github.hiroshitsu.qcalc, the same as the Java package) to the manifest, compiles the resources with aapt2, and makes the base of the APK with the manifest and the page (app/assets). When linking, --rename-manifest-package changes the package name to io.github.hiroshitsu.qcalc.debug, and aapt2 rewrites .MainActivity to the full name io.github.hiroshitsu.qcalc.MainActivity (as Gradle's applicationIdSuffix does). If io.github.hiroshitsu.qcalc.debug were written in the manifest itself, Android would look for io.github.hiroshitsu.qcalc.debug.MainActivity, which does not exist, and the app would not start. After linking, aapt2 dump badging checks that the activity started is io.github.hiroshitsu.qcalc.MainActivity, and the script stops if it is not. Linking uses the API 34 android.jar. minSdk is 24 and targetSdk 36; versionCode and versionName are read from version.properties, with -debug added to versionName. The source manifest has no package attribute because Gradle takes the package name from namespace.
3. Compiles the Java (the 5 files in app/src; PlayBilling is not included) against the API 36 android.jar (-source 8 -target 8) and makes classes.dex with dx (build_dex in build-common.sh, shared with the App Bundle).
4. Adds the engine (app/jniLibs/arm64-v8a/libqalcengine.so) as lib/arm64-v8a/libqalcengine.so: Android extracts only the `lib*.so` files in lib/ to a place where they can be run (nativeLibraryDir), together with extractNativeLibs="true" in the manifest.
5. Aligns with zipalign (4-byte boundaries, -p) and signs with apksigner (v1, v2 and v3). The result is out/QCalc-1.3.0-debug.apk (3,315,796 bytes in 1.3.0).

The test key

- signing/qcalc-debug.keystore (alias debug). If it is missing, keytool makes it in the first build (RSA 3072 bits, valid for 10,000 days, CN=QCalc debug). Its password is written in build-apk.sh, as the key is only for the test build.
- It is used only for the test build. The package name and the key differ from the Google Play version, so both can be installed side by side without replacing each other.
- After the key is made again, the installed test build can no longer be updated (uninstall the test build first).

Commands to check

```
apksigner verify --verbose --print-certs out/QCalc-1.3.0-debug.apk   #
signature and certificate
aapt2 dump badging out/QCalc-1.3.0-debug.apk                         #
package name, version, SDK, permissions, activity started
unzip -l out/QCalc-1.3.0-debug.apk                                   #
contents (lib/arm64-v8a/libqalcengine.so and the rest)
```

5.6 Making the App Bundle for Google Play (build-aab.sh)

An App Bundle (.aab) is uploaded to Google Play. build-aab.sh puts the .aab together without Gradle or bundletool and signs it with the upload key (its contents are in 3.17 of the detailed design).

1. Adds the package attribute (io.github.hiroshitsu.qcalc) to the manifest and makes the manifest and the resource table (resources.pb) in protobuf format with aapt2 link --proto-format (the API 34 android.jar; minSdk 24, targetSdk 36, versions from version.properties).
2. Compiles the Java against the API 36 android.jar and makes classes.dex (as for the test APK).
3. Zips BundleConfig.pb (bundletool 1.17.2, with the “uncompress native libraries” optimization turned off) with base/manifest, base/dex, base/res, base/assets and base/lib/arm64-v8a/libqalcengine.so (the time of each entry fixed at 1981-01-01).
4. Signs with the upload key using jarsigner (SHA-256, SHA256withRSA) and checks with jarsigner -verify. The result is out/QCalc-1.3.0.aab (3,304,203 bytes; the SHA-256 of 1.3.0 is 71d5f5d83ca313fde21672db30edd82aec0c3554ec0bb2c98b733955056e5b23).

The App Bundle of build-aab.sh does not include PlayBilling, so QCalc Pro cannot be bought (billing is none) and all the Pro features can be used. An App Bundle in which Pro can be bought is made with Gradle (5.11).

The upload key

- signing/qcalc-upload.jks (alias upload). The password is read from the environment variable QCALC_UPLOAD_PASS, or from the first line of the file signing/qcalc-upload.pass. The place and the alias can also be changed with QCALC_UPLOAD_KEYSTORE and QCALC_UPLOAD_ALIAS.
- With Play App Signing, the APKs that reach users′ devices are signed with the app signing key that Google manages. The upload key is used only by Google Play, to check that an upload comes from the developer.

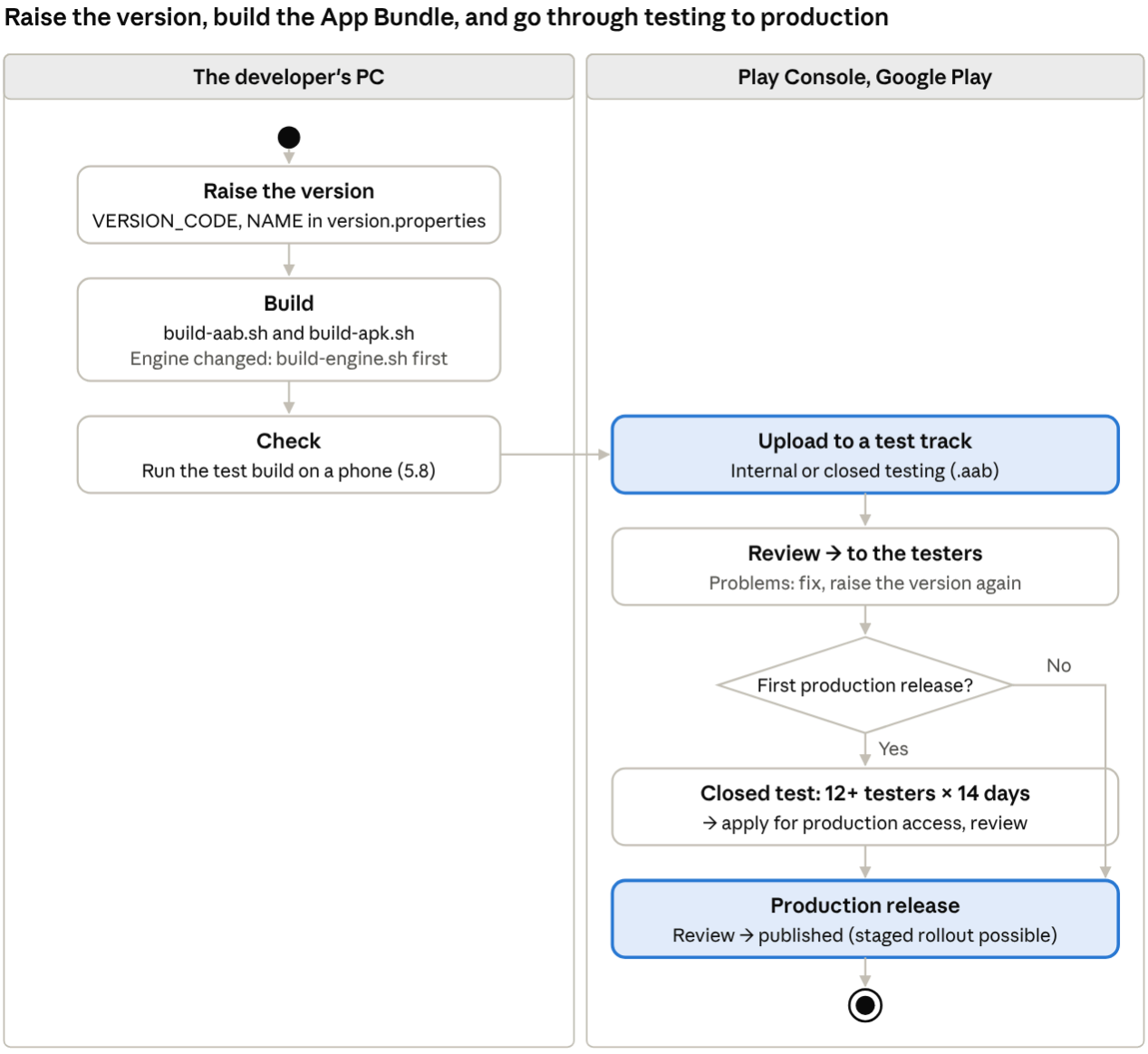
- The key and its password are never published (signing/ is excluded by .gitignore). Keep a copy in a safe place. If it is lost or leaked, ask for an upload key reset in Play Console (no update can be published until the new key is registered).

Commands to check (bundletool from https://github.com/google/bundletool/releases)

```
jarsigner -verify out/QCalc-1.3.0.aab                                    #
checks the signature
keytool -printcert -jarfile out/QCalc-1.3.0.aab                          # the
upload key's certificate (the same as in Play Console?)
java -jar bundletool.jar validate --bundle=out/QCalc-1.3.0.aab
java -jar bundletool.jar dump config --bundle=out/QCalc-1.3.0.aab        #
uncompressNativeLibraries must not be enabled
java -jar bundletool.jar build-apks --bundle=out/QCalc-1.3.0.aab --
output=qcalc.apks --device-spec=phone.json
```

5.7 Publishing on Google Play and updating

The detailed steps of the first release (the fields in Play Console, the data safety form, finding testers and so on) are in a separate publishing guide (in Japanese). Here are the flow and the steps for updating a version.



Activity diagram · publishing and updating

Updating the version

1. Make VERSION_CODE in version.properties larger than the previous version's (for example 5 → 6; 1.0.0 was 1, 1.1.0 was 2, 1.2.0 was 3 and 1.3.0 is 5, as 4 was used by a 1.3.0 App Bundle made before English became the default), and change VERSION_NAME (for example to 1.3.1).
2. After changing only the page or Java, run ./build-aab.sh; after changing the engine, run ./build-engine.sh and then ./build-aab.sh.
3. Make the test build too with ./build-apk.sh, and do the checks of 5.8 on a phone.
4. In Play Console, choose a track (internal testing, closed testing or production), make a new release, upload the .aab, write the release notes and send it for review.
5. When it passes the review, it is delivered. In production, a staged rollout (delivering to a growing share of users) can be used too.

- A VERSION_CODE can never be used twice (an .aab with a used value is not accepted). Always use the same upload key.
- A new personal developer account needs a closed test with at least 12 testers taking part for 14 days in a row before applying for production (Google Play's rule as of November 2026).
- Raise targetSdk to follow Google Play's rule (a newer API level is required from August 31 each year). QCalc 1.3.0 also uses 36.
- After changing features or the network services used, review the privacy policy (docs/privacy.html), the data safety form and the store listings (store/listing-en.md, listing-ja.md). Update the sources in the repository to match the version distributed (GPL).
- Before publishing a version that sells QCalc Pro, create and activate the in-app product (a one-time product) qcalc_pro in Play Console, and upload an .aab made with Gradle (qcalc.billing=true; 5.11). The .aab of build-aab.sh is a version without billing, and people who used it can use Pro for free even after updating to a version with billing (legacy; detailed design 3.20).

5.8 How to check that it works

Running only the engine on a PC

Run the arm64 engine with qemu and give it JSON, one line at a time.

```
echo '{"id":1,"cmd":"calc","expr":"integrate(x^2 sin(x), 0, pi)"}' |
QALCULATE_USER_DIR=/tmp/q qemu-aarch64-static bridge/qalc-engine
echo '{"id":2,"cmd":"plot3d","exprs":["sin(x) cos(y)"],"nx":5,"ny":5}' |
QALCULATE_USER_DIR=/tmp/q qemu-aarch64-static bridge/qalc-engine
echo '{"id":3,"cmd":"calc","expr":"[1, 2; 3, 4] × [5, 6; 7, 8]"}' |
QALCULATE_USER_DIR=/tmp/q qemu-aarch64-static bridge/qalc-engine
```

The result is one line that contains `"result":"π² − 4 ≈ 5.869604401"`. With `"store":true`, a line for the ans event follows. plot3d returns one line with x, y and series (z for each expression). Matrices come back as one line of text such as `"result":"[19 22; 43 50]"` (rows separated by "; "), which the page lays out as a table. To compare with the official qalc, build the same tag of libqalculate for a PC, calculate with qalc -defaults and put the results side by side (6.2).

Running the page on a PC

During development, a small HTTP server served app/assets and passed the requests for the engine to the engine running in qemu. A window.QalcDev (which forwards send to POST /engine) is put into the page, which is shown in Chromium (Playwright) and

operated automatically. Exchange-rate updates are simulated by this server. The pictures of the user manual and the screenshots in store/ were also taken with this tool. These test tools are not in the repository.

Checking on a phone

```
adb install -r out/QCalc-1.3.0-debug.apk
adb logcat -s QCalc      # JavaScript errors of the page and more
(MainActivity logs with the tag QCalc)
```

What to check: starting and calculating; the 4 tabs of keys (123, Functions, Calculus, Matrix) and typing templates; matrix results shown as tables; the examples; Undo; text sizes; reusing the history; the history, the expression being typed and ans staying after closing and opening the app; updating exchange rates; 2D and 3D graphs (moving, turning, zooming, tapping); user definitions; switching dark mode; turning the screen; Back (gestures and predictive back); on Android 15 and later, the page not overlapping the status bar, the navigation bar or the keyboard; About and the open source licenses; TalkBack; the language (starting in English, and switching with Language / 言語); color themes; exporting the history (CSV); the QCalc Pro panel and the PRO marks (the test build has Pro from the start, so turn off “Pro unlocked (for testing)” in the Pro panel to check). How the APKs that Google Play delivers behave can be checked by uploading the .aab to internal testing and installing it from Google Play.

5.9 Updating to a new libqalculate

1. Check the new upstream tag (https://github.com/Qalculate/libqalculate) and its changes.
2. Put the new source (a tar.xz made from the tag with git archive) in third_party/ and remove the old one. Update the version, tag and commit in third_party/README.md too.
3. Change the version numbers (the table below).
4. Delete prefix/lib/libqalculate.a and the old folder in src/, then run ./fetch-sources.sh and ./build-engine.sh.
5. Fix the compile errors and warnings of qalc-engine.cc. calculate_once and store_ans follow qalc's calculation steps (Calculator::calculateAndPrint and qalc.cc), so check the upstream diff for changes there and follow them. Also check how expressionToPlotVector and generateVector, used by plot and plot3d, behave. The Japanese units and trace() (add_japanese_units, add_matrix_functions) are not added when units or functions of the same names come upstream, so check which

ones are used. Closing message holds (close_message_stops) can stay even if upstream fixes the problem, but check with tests M03 to M05 that the warnings appear (detailed design 3.19).

6. Check with getExchangeRatesUrl and getExchangeRatesFileName that the exchange-rate sources and file formats have not changed, and fix the checks of RatesUpdater if needed.
7. If there are new category names or messages, update CATEGORY_JA, MESSAGES_JA and HIDDEN_ITEMS in app.js. Add the Japanese names of new functions, constants and units to FUNC_JA, VAR_JA and JA_NAMES. Run all the examples (EXAMPLES) through the engine in the tests and check that their results have not changed.
8. Make the license texts again with python3 make-licenses.py (if the copyright years changed, fix the copyright in make-licenses.py too).
9. Build with ./build-apk.sh and ./build-aab.sh and do the tests of chapter 6 again (including the comparison with the official qalc). Publish on Google Play with the steps of 5.7.

File	What to change
fetch-sources.sh	The tag of git clone (--branch v5.13.2), the file name (libqalculate-5.13.2.tar.xz) and the folder name
third_party/README.md	Version, tag, commit
build-engine.sh	The folder name src/libqalculate-5.13.2 (2 places) and -DQALC_ENGINE_VERSION='"5.13.2"'
make-licenses.py	The folder name and version of libqalculate ('5.13.2')
version.properties	VERSION_NAME (for example 1.3.1) and VERSION_CODE (always larger than before)
app/src/io/github/hiroshitsu/qcalc/RatesUpdater.java	The libqalculate version in the User-Agent (“QCalc/〈app version〉 (Android; libqalculate 5.13.2)”; the app version comes from versionName automatically)
README.md, README.ja.md	The version (libqalculate 5.13.2)

When updating GMP, MPFR or libxml2, change the file in third_party/ and SHA256SUMS, the URL and file name in fetch-sources.sh, the folder name in build-engine.sh, and the version and folder name in make-licenses.py; then delete the .a of that library in prefix/lib and of the libraries that depend on it (finally libqalculate.a) before building.

5.10 Notes for maintenance

Item	What to watch
Upload key	Never publish it. Keep a copy in a safe place. If it is lost or leaked, ask for a reset in Play Console. Google manages the app signing key, so even if the upload key is lost, users' apps can still be updated
versionCode	Larger for every release (VERSION_CODE in version.properties). A value used once cannot be used again
Precision limit (200000 digits)	Keep MAX_PRECISION the same in app/assets/app.js and bridge/qalc-engine.cc. When changing it, also adjust MAX_RESULT in app.js (the largest result in bytes when committing) and MAX_SAVED_ANSWER in Engine.java (the length of the ans saved), and check that results fit the limit of Android's clipboard (about 1 MB at a time) and that sin, ln and the like finish within the commit time limit (20 seconds) (H01 to H16 in 6.2 of the test specification). The limit without QCalc Pro (1,000 digits) is FREE_MAX_PRECISION in app.js
targetSdk	36. Raise it every year to follow Google Play's rule, and check the behavior changes of that version (screen edges, Back and so on; detailed design 3.15)

Item	What to watch
The engine's file name and extraction	Do not change the name lib/arm64-v8a/libqalcengine.so, extractNativeLibs="true", or the setting that turns off uncompressNativeLibraries for the App Bundle (useLegacyPackaging = true in Gradle). From Android 10, files in the app's data folder cannot be run, so the engine is packed so that it is extracted to nativeLibraryDir
16 KB pages	Keep -Wl,-z,max-page-size=16384 in the engine's link (Google Play's rule)
Which android.jar	Java uses the API 36 android.jar, aapt2 the API 34 one. Use new APIs only after checking Build.VERSION.SDK_INT (minSdk is 24)
The boundary between the page and Java	Only 14 methods are exposed in QalcNative (1.3.0 added the 5 of QCalc Pro and saveFile). Check the arguments of any method added too (StateStore names, kinds of URL and so on). Keep the WebView's file access turned off
Exchange-rate sources	The URLs and file names come from libqalculate, but the checks and the backup URL of the third source are written in RatesUpdater. Fix them if the formats of the sources change
What QCalc adds to libqalculate	The 10 Japanese units, trace() and closing message holds are in qalc-engine.cc, not in libqalculate's source (detailed design 3.19; "Differences from libqalculate" in the README). Keep libqalculate's source unchanged, and check with the comparison tests that results are the same as those of the official qalc
Licenses	The app is under GPL v2 or later, the engine as a whole under GPL v3 or later (as MPFR is under LGPL v3 or later). When a library changes, change third_party/ and licenses.js too, and keep publishing in the repository the same sources as the version distributed. QCalc's own code carries an exception that allows linking with the Google Play Billing Library and distributing the result (License in the README)

Item	What to watch
Privacy	Designed to collect no personal information. Before adding analytics, ads or a new network service, update the privacy policy (docs/privacy.html), the explanation in the app and the data safety form first
Test tools	The test scripts used in development (chapter 6) are not in the repository. Adding them would help when the tests are repeated for maintenance
QCalc Pro (billing)	Versions in which Pro can be bought are built with Gradle (qcalc.billing=true; 5.11). PlayBilling uses Google Play Billing Library 8. When raising the library version, check removed APIs and the acknowledgement of purchases (within 3 days). Before changing LEGACY_USERS_KEEP_PRO (earlier users get Pro for free), first decide how to treat people who used a version without billing (detailed design 3.20)
Language of the screen	Write new texts of the page in both languages with tr(English, Japanese). Keep the texts of the engine and Java in English, and add the ones to show in Japanese on the Japanese screen to MESSAGES_JA. English is the default; Japanese only when the user chose it (qalc.lang; detailed design 3.21). Check that no Japanese is left on the English screen (chapter 6)

5.11 Opening in Android Studio

Settings are included so that the same sources can be opened in Android Studio (Gradle), built and run on a phone. The engine is not built: the built one in app/jniLibs is included as it is.

Item	Contents
What is needed	Android Studio for Android Gradle Plugin 8.11.1. Gradle 8.13 and Android SDK Platform 36 are downloaded when the project is first opened, following the prompts

Item	Contents
Opening	Choose the top folder of the repository (where settings.gradle is) with “Open”. After syncing, the app module appears
Where the sources are	sourceSets in app/build.gradle point to the same folders as build-apk.sh (app/src, app/res, app/assets, app/jniLibs). The version is read from version.properties
Debug build	Installed with Run (▶). The package name is io.github.hiroshitsu.qcalc.debug, and the name “QCalc (debug)” (app/res-debug). It installs as an app apart from the Google Play version, with its own history and settings
For Google Play	In the Build menu, “Generate Signed App Bundle or APK…”, choose Android App Bundle and give the upload key signing/qcalc-upload.jks (alias upload). With keystore.properties (copied from keystore.properties.example and filled in), release builds are signed automatically
Version with billing	Setting qcalc.billing to true in gradle.properties (or -Pqcalc.billing=true) includes PlayBilling from app/src-billing and Google Play Billing Library 8.3.0 (from Google's Maven repository), which makes a version in which QCalc Pro can be bought. With the default false there is no billing, as in the versions made by the scripts, and all the Pro features can be used
Differences from the scripts	compileSdk 36 and targetSdk 36, and the dex is made with D8. The engine is packed compressed and extracted when the app is installed (useLegacyPackaging = true)

Notes

- Before uploading an .aab made with Gradle, check with bundletool dump config that uncompressNativeLibraries is not enabled (no "enabled": true). If it is, the engine is not extracted in the APKs that Google Play makes, and nothing can be calculated.
- gradlew and gradle-wrapper.jar for the command line are not included. Android Studio needs only gradle/wrapper/gradle-wrapper.properties.

- After changing qalc-engine.cc or libqalculate, make the engine in app/jniLibs again by running build-engine.sh on Linux (WSL on Windows).
- These settings were made where Gradle and the Android SDK could not be downloaded, so building in Android Studio has not been checked yet (chapter 6). For the same reason, the version with billing (qcalc.billing=true) has not been built, and no purchase has been tried. Check purchases with a version installed from internal testing, using an account registered as a license tester in Play Console.